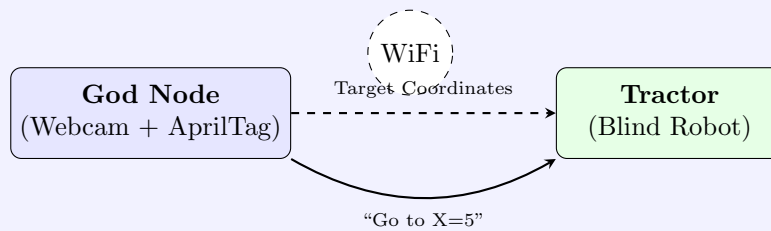


ROS2 Spring Workshop

Lab 5: The Global Eye (Vision, Calibration & Transforms)

Project Context: The God Node Architecture

The Goal: We are building a distributed system. The Camera (Server) stays fixed on the ceiling and tells the Tractor (Client) where it is.



Objective

In previous labs, we relied on `/turtle1/pose` to tell us exactly where the robot was. This was “easy mode”: perfect, noiseless data. In the real world, robots must use sensors to *calculate* where they are.

In this lab, you will build the “eye” of the God Node proposed in your Final Project. You will learn three things that every real robotics vision system needs:

1. How to detect and identify visual markers (AprilTags).
2. How to **calibrate** a camera so the math is actually accurate.
3. How to read 3D pose information out of a transform tree.

You may use any USB webcam or your laptop’s integrated camera.

Pre-Lab Concept Primer

1. What is an AprilTag?

An AprilTag is a visual fiducial system (like a QR code for robots). It allows a camera to determine the precise 3D position (XYZ) and orientation (roll-pitch-yaw) of the tag relative to the lens.

Why “36h11”? You will see this code often. AprilTags come in “families”:

- **36:** The tag has a 6x6 bit grid (36 bits of data).
- **h11:** It has a “Hamming distance” of 11. This means the tag is extremely robust; even if 10 bits are covered by mud or glare, the robot can still read it mathematically.

2. The Transform Tree (TF)

Standard cameras are “optical frames”:

- **Z-axis:** Points *forward* (out of the lens).
- **X-axis:** Points *right*.
- **Y-axis:** Points *down*.

This is weird! In our map (world frame), **Z** is usually *up*. ROS 2 uses the **TF2 library** to handle this conversion automatically, and AprilTag has a `z_up` parameter that we’ll use to match the world convention.

3. Why Calibration Matters (The Pinhole Model)

A camera's math assumes a **pinhole model**: light enters through one point and projects onto a flat sensor. Real lenses bend light. If we don't tell the software exactly how YOUR specific lens bends light, every measurement will be off by 5–20%.

Two numbers matter most:

- **Focal length:** How “zoomed in” the lens is, in pixels.
- **Distortion coefficients:** How the lens curves straight lines.

We find these by showing the camera a **checkerboard of known geometry** from many angles, and letting OpenCV solve for the lens parameters that make all those checkerboard corners line up. This is what you'll do in Part 2.

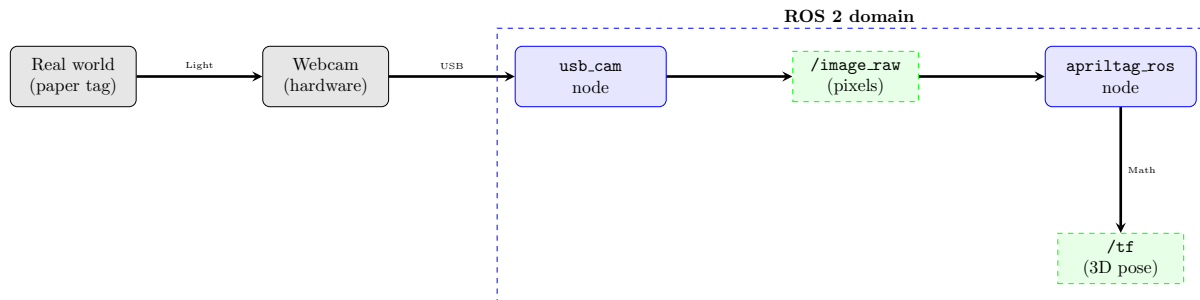


Figure 1: The Vision Pipeline: how reality gets converted into math.

Phase 0: The Hardware Handshake

Linux users: Your USB camera should already appear as `/dev/video0` automatically. **Skip directly to Part 1.**

macOS Users: Read This Before You Start

Docker Desktop on macOS **does not reliably support USB device passthrough** to containers. This is a hypervisor-level limitation, not something XQuartz or a different webcam can fix. (Docker Desktop 4.35+ added experimental USB/IP support, but macOS lacks a mature USB/IP server, so it's not viable for a 2-hour lab.)

Your options for Labs 5 and 6:

1. **Pair up** with a classmate on Windows or Linux. This is the recommended path. You can still write and review the code on your own laptop; just run the camera-touching parts on theirs.
2. **Use a lab-room machine** if your instructor has Linux workstations available.

Optional Mac sanity check: If you want to know whether your specific Docker Desktop version supports USB passthrough (it's evolving), open the Dev Container terminal and run `ls /dev/video*`. If you see `/dev/video0`, congratulations!!! you can do Labs 5 and 6 solo. If you get “No such file or directory,” plan to pair up. The check takes 5 seconds.

Docker on Windows cannot see your webcam by default. Windows “hogs” it. You must detach it from Windows and plug it into the Linux kernel (WSL 2).

Do this in PowerShell (Admin)

1. **Install the tool:** `winget install dorssel.usbipd-win` (then restart your computer).
2. **Find your Bus ID:** `usbipd list` (e.g., let's say it is 1-13).
3. **Bind it:** `usbipd bind --busid 1-13`
4. **Attach to Docker:** `usbipd attach --wsl --busid 1-13`

Note: If you get an error, ensure Docker Desktop is running first.

Verification: Inside your VS Code terminal, run `ls -l /dev/video*`. If you see yellow text (`/dev/video0`), you are ready to proceed.

Part 1: Package Setup

Before we configure the camera, we need a home for our vision logic.

1. **Go to the source folder:**

```
1 cd /workspaces/agtech_ros2/src
2
```

2. **Create the package:** We use `ament_cmake` for this lab because it handles configuration files better than Python packages.

```
1 ros2 pkg create --build-type ament_cmake lab5_vision
2
```

3. **Create the config folder:** We need a place to store our tag settings.

```
1 mkdir -p /workspaces/agtech_ros2/src/lab5_vision/config
2
```

4. **Copy the tag config:** We have provided the `tags.yaml` file for you in the course resources.

```
1 cp /workspaces/agtech_ros2/lab_resources/lab5/tags.yaml \
2   /workspaces/agtech_ros2/src/lab5_vision/config/
3
```

Part 2: Real Camera Calibration

This is where most online tutorials cheat — they hand you a “dummy calibration file” and your distance measurements come out 5–20% wrong. We’re not doing that. You’re going to calibrate your actual camera with a checkerboard, and your measurements will be accurate to 1–2%.

The Calibration Board

Your instructor has printed a **checkerboard calibration target** and distributed one to each group. It is a 9×7 grid of black and white squares — that means 8×6 **internal corners** (the points where 4 squares meet).

Each square should be **25 mm** on a side. **Verify with a ruler** before you calibrate — if your board printed at 95% scale, the math will be wrong by 5%.

Important vocabulary for the command below:

- **Squares** = the black/white tiles (9 wide, 7 tall)
- **Inner corners** = the points where 4 squares meet (8 wide, 6 tall)
- The calibrator asks for **inner corners**, not squares. This trips up everyone.

Step 1: Install the Calibration Tool

```
1 sudo apt install ros-humble-camera-calibration \
2                 ros-humble-usb-cam \
3                 ros-humble-apriltag-ros
```

Step 2: Start the Camera

In **Terminal 1**, start the `usb_cam` driver:

```
1 ros2 run usb_cam usb_cam_node_exe --ros-args \
2   -p video_device:=/dev/video0 \
3   -p pixel_format:="mjpeg2rgb"
```

You'll see some warnings about "camera not calibrated" — that's fine, we're about to fix that.

Step 3: Run the Calibrator

In **Terminal 2**:

```
1 ros2 run camera_calibration cameracalibrator \
2   --size 8x6 --square 0.025 \
3   --ros-args -r image:=/image_raw -r camera_info:=/camera_info
```

Decoding the arguments:

- `--size 8x6` is **inner corners**, not squares. Your board has 8×6 inner corners.
- `--square 0.025` is the square edge length in **meters** (25 mm).

Step 4: Collect Calibration Data

A GUI window appears showing your camera feed. On the right, you'll see four progress bars: **X**, **Y**, **Size**, and **Skew**.

Your job: hold the checkerboard in front of the camera and move it around until all four bars fill up and the **CALIBRATE** button lights up green.

Tips for fast calibration:

- **X**: Move the board left and right across the whole frame.
- **Y**: Move it up and down.
- **Size**: Move it close to the camera (filling most of the frame) and far (small in the frame).
- **Skew**: Tilt the board at various angles — forward, backward, sideways. This is usually the hardest to fill.
- Good lighting helps the detector lock onto corners. Avoid harsh shadows on the board.

You'll need about 40 detected views. This takes 2–3 minutes.

Step 5: Commit the Calibration

Once **CALIBRATE** is green, click it. The tool will think for 30–60 seconds (doing math), then the **SAVE** and **COMMIT** buttons light up.

Click **SAVE** (writes a tarball to `/tmp`), then **COMMIT** (publishes the calibration to the camera node).

Step 6: Install the Calibration Permanently

The calibration is now published, but lives only in memory. Close the calibrator window. The tarball at `/tmp/calibrationdata.tar.gz` contains a file called `ost.yaml` — that's your calibration.

Extract it and install:

```
1 # Unpack the calibration tarball
2 cd /tmp
3 tar -xzf calibrationdata.tar.gz
4
5 # Make the config folder in the right place
6 mkdir -p ~/.ros/camera_info
```

```

7
8 # Rename ost.yaml to match the camera_name (usb_cam's default is "default_cam")
9 # Open ost.yaml and find the line "camera_name: narrow_stereo"
10 # Change it to: camera_name: default_cam
11 # Then save as ~/.ros/camera_info/default_cam.yaml:
12 cp /tmp/ost.yaml ~/.ros/camera_info/default_cam.yaml

```

The Name-Matching Trap

The calibrator saves the file with `camera_name: narrow_stereo` by default (a historical default from a stereo camera rig). The `usb_cam` node looks for a file whose `camera_name` matches `default_cam`. **If these do not match, the driver silently ignores your calibration** and you'll wonder why nothing changed.

Open `/.ros/camera_info/default_cam.yaml` in VS Code and change:

```
camera_name: narrow_stereo
```

to:

```
camera_name: default_cam
```

Save the file.

Step 7: Sanity-Check Your Calibration

Look at the YAML file. The `camera_matrix` should have values that look like this:

```

camera_matrix:
  data: [900ish, 0, 960ish, 0, 900ish, 540ish, 0, 0, 1]

```

The first and fifth numbers (focal length in X and Y) should be within 5% of each other — if they're not, your calibration is bad and you should redo it with more diverse views.

Distortion coefficients should be small numbers (under 0.5 for the first two). If `k1` is larger than 1.0 in absolute value, something went wrong.

Part 3: Running the Camera with Your Calibration

Now restart the camera, this time pointing it at your calibration file:

```

1 ros2 run usb_cam usb_cam_node_exe --ros-args \
2   -p video_device:=/dev/video0 \
3   -p pixel_format:="mjpeg2rgb" \
4   -p camera_name:=default_cam \
5   -p camera_info_url:="file:///root/.ros/camera_info/default_cam.yaml"

```

This time, no “camera not calibrated” warning.

Syntax Spotlight: Anatomy of a ROS 2 Command

Let's decode the “magic spell”:

- `ros2 run`: The verb. We're running a program.
- `usb_cam`: The **package** (the library folder).
- `usb_cam_node_exe`: The **executable** (the actual script inside the folder).
- `--ros-args`: *“Everything after this is for ROS, not Linux.”*
- `-p`: Sets a **parameter** (configuration variable).
- `pixel_format:="mjpeg2rgb"`: We force the node to use MJPEG compression to save USB bandwidth, then unzip to RGB for ROS.

Part 4: The Detector (AprilTag Node)

Now we convert “pixels” into “pose”.

Critical: Do NOT use your phone screen!

You must use a **printed paper tag**.

- **Why?** Computer vision algorithms look for sharp contrast. A phone screen emits light and has a glass surface that reflects your room lights (specular reflection). This blinds the camera.
- **Get the tag:** <https://chaitanyantr.github.io/apriltag.html>
- **Instruction:** Select family **tag36h11** and ID **0**.

Measure Your Tag's Black Square

The `size` parameter in `tags.yaml` is **the edge length of the BLACK SQUARE in meters** — not the whole printed image, not the page, just the black part.

Grab a ruler. Measure the black square on your printed tag. A typical print on letter/A4 paper is around 12 cm (0.12 m), but yours might be different.

Open `tags.yaml` and set `size:` to the value you measured. If you don't, your distance estimates will be wrong by the same ratio.

1. **Edit your config file:** Open `tags.yaml`. We've added performance parameters for you to see.

```
1 apriltag:
2   ros__parameters:
3     family: 36h11
4     size: 0.12           # EDGE LENGTH OF THE BLACK SQUARE (measure it!)
5     max_hamming: 0
6     z_up: true           # Report tag Z as depth into the scene
7     # --- PARAMETERS TO TUNE ---
8     decimate: 2.0        # 1.0 = full res, 2.0 = half res (faster)
9     threads: 4           # Use more CPU cores
10    refine_edges: 0       # 1 = precise, 0 = fast
11
```

What does `z_up: true` actually do?

Without `z_up: true`, the detector reports tag pose in the camera's *optical* frame where Z points forward out of the lens. Distance to the tag ends up on the **Z axis**, which is counterintuitive when visualized in RViz.

With `z_up: true`, the reported pose is rotated so that Z points up (matching the world convention). Depth into the scene now shows up on the axis your brain expects. Your "Reality Gap" challenge at the end measures distance along the camera's optical Z axis either way — but with `z_up: true`, the numbers match the way humans naturally think about space.

2. **Run the node:**

```
1 ros2 run apriltag_ros apriltag_node --ros-args \
2   -r image_rect:=/image_raw \
3   -r camera_info:=/camera_info \
4   --params-file /workspaces/agtech_ros2/src/lab5_vision/config/tags.yaml
5
```

About That `image_rect` Remap

We map the topic `image_rect` (which suggests a pre-rectified image) to `/image_raw` (which is NOT pre-rectified). This works because AprilTag applies distortion correction internally using `camera_info`.

In Lab 9 we'll add a proper `rectify_node` in between for better accuracy at the edges of the frame. For today, the internal correction is fine.

Part 5: Verification

Before opening RViz, do a quick terminal check. In a new terminal:

```
1 ros2 topic echo /tf --once
```

You should see a transform with `child_frame_id: tag36h11:0`. If you see this, detection is working. RViz is optional confirmation — the detector doesn't need it.

RViz Visualization

1. Open RViz: `rviz2`
2. The “ghost frame” fix: In the top-left “Global Options”, the **Fixed Frame** dropdown might be empty because ROS hasn't seen the camera yet. **Manually type:** `default_cam` and hit Enter.
3. Add visuals:
 - Click **Add** and select **Camera** (not just “Image”!).
 - Set the **Topic** to `/image_raw`.
 - *Why? The “Camera” display type automatically overlays the 3D TF arrows on top of the 2D video feed.*

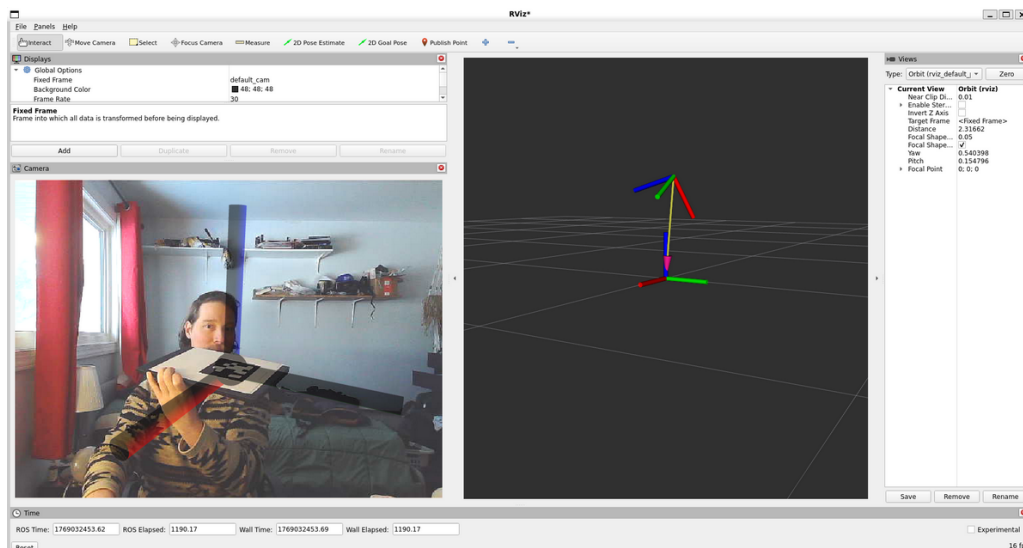


Figure 2: Expected result: the TF axes should appear firmly attached to the physical tag in the video feed.

Part 6: Challenge — The Reality Gap (and Why Your Calibration Matters)

Robots trust their sensors, but sensors lie. You already did the work to shrink that lie: real calibration. Let's measure how well it paid off.

1. Place your AprilTag flat on a desk.
2. Use a tape measure to position the camera **exactly 50 cm (0.50 m)** from the tag. Measure from the front of the lens to the centre of the tag.
3. In RViz, read the tag's reported position. Or in a terminal:

```
1 ros2 topic echo /tf --once
2
```

Look for translation: `z: ...` (with `z.up: true`, this is the depth into the scene).

4. Compute your percent error:

$$Error\% = \frac{|Measured - 0.50|}{0.50} \times 100\%$$

5. Now try 1.0 m and 1.5 m. Record all three.

Interpreting Your Error

- **Under 2%:** Excellent. Your calibration is dialed in. You're getting accuracy that rivals commercial indoor positioning systems.
- **2–5%:** Good. Normal for this class of camera. Slight misalignment in your calibration data or tag measurement.
- **5–10%:** Usable but worth investigating. Most likely your tag **size** parameter is off — remeasure the black square.
- **Over 10%:** Something's wrong. Likely suspects: tag size way off, or your calibration used too few views / poor pose variety. Consider recalibrating.

Remember: Before you calibrated, a dummy calibration would have produced 10–20% errors routinely. You fixed that by doing this properly. The rest of the course (God Node, Labs 9–10) relies on this foundation.

Pro Tip: Is Your Laptop Lagging?

The detector math is heavy. Here's how to reduce load:

1. In `tags.yaml`, set `decimate: 2.0`. This halves the resolution the detector uses. About 4x faster but might miss small tags.
2. **Close RViz.** RViz rendering can eat 30% of a CPU core. The detector itself runs at the same rate either way — RViz is just a viewer. If you need to verify, use `ros2 topic echo /tf --once` in a terminal.
3. Increase `threads`: if your CPU has 8+ cores. Default 4 is fine for most laptops.

Cleanup Tip

If `Ctrl+C` on `usb_cam` hangs or leaves the device locked, wait 5 seconds for the kernel to release it. If that fails on Linux:

```
fuser -k /dev/video0
```

This force-releases any process still holding the camera.

Deliverables

Submit a single PDF containing:

1. **Calibration proof:** Your `default_cam.yaml` contents (paste the YAML into your document). Specifically flag:
 - Your focal length values (`fx`, `fy` from the `camera_matrix`)
 - Your `k1` distortion coefficient
 - Whether `fx` and `fy` are within 5% of each other (a sanity check)
2. **Detection screenshot:** RViz showing the TF axes attached to your tag.
3. **Reality Gap measurements:** A table with three rows (50cm, 1m, 1.5m), each showing physical distance vs. reported Z vs. percent error.
4. **Analysis (3–5 sentences):** Did your error increase, decrease, or stay constant with distance? What's the most likely source of residual error in your system — calibration, tag size measurement, or the detector itself? Briefly justify.